Phase 1 Notes: Recursion to Recurrence Relations

1. Main Goal of Phase 1

In this phase, we learn how to look at a recursive function and write a mathematical equation for its running time.

That mathematical equation is called a **recurrence relation**.

The main idea is:

> Recursive code can be converted into a formula that shows how much time the code takes.

In this phase, we use a simple example:

```
void printNumber(int n) {
    if (n > 0) {
        cout << n << " ";
        printNumber(n - 1);
    }
}
```

The recurrence relation for this function is:

```
T(n) = T(n - 1) + 1
T(0) = 1
```

The final time complexity is:

```
O(n)
```

---

2. What Is Recursion?

A function is called **recursive** when it calls itself.

Example:

```
void printNumber(int n) {
    if (n > 0) {
        cout << n << " ";
        printNumber(n - 1);
    }
}
```

Here, the function `printNumber` calls itself here:

```
printNumber(n - 1);
```

So this is a recursive function.

---

3. How the Recursive Function Works

Suppose we call:

```
printNumber(5);
```

The function starts with `n = 5`.

It checks:

```
if (n > 0)
```

Since `5 > 0` is true, it prints `5`, then calls:

```
printNumber(4);
```

Then the same thing happens again.

The calls happen like this:

```
printNumber(5)
printNumber(4)
printNumber(3)
printNumber(2)
```

```
printNumber(1)
printNumber(0)
```

When `n = 0`, the condition becomes false:

```
0 > 0
```

This is false, so the function stops.

The output is:

```
5 4 3 2 1
```

---

4. What Is a Recurrence Relation?

A **recurrence relation** is a mathematical equation used to describe the running time of a recursive function.

It tells us how the time for a bigger input depends on the time for a smaller input.

Instead of asking:

> What does the function print?

we ask:

> How much work does the function do?

For recursive functions, the total time usually has two parts:

1. The work done in the current function call.
2. The time taken by the recursive call.

---

5. Meaning of `T(n)`

We use `T(n)` to mean:

> The time taken by the function when the input size is `n`.

Examples:

```
T(5)
```

means:

> Time taken when `n = 5`.

```
T(100)
```

means:

> Time taken when `n = 100`.

So `T(n)` does not mean the output of the function.

It means the running time of the function.

---

6. The Main Example Code

```cpp
#include <iostream>
using namespace std;

void printNumber(int n) {
    if (n > 0) {
        cout << n << " ";
        printNumber(n - 1);
    }
}

int main() {
    printNumber(5);
    return 0;
}
```

This function prints numbers from `n` down to `1`.

For example:

```
printNumber(5);
```

prints:

```
5 4 3 2 1
```

---

7. Breaking the Function Into Parts

Look at the important part of the function:

```
if (n > 0) {
    cout << n << " ";
    printNumber(n - 1);
}
```

Inside each call, two things happen.

Part 1: Current Work

This line prints one number:

```
cout << n << " ";
```

Printing one number takes constant time.

Constant time is written as:

```
1
```

So the current work is:

```
+1
```

Part 2: Recursive Call

This line calls the same function again:

```
printNumber(n - 1);
```

The input becomes smaller.

It changes from:

```
n
```

to:

```
n - 1
```

So the time taken by the recursive call is:

```
T(n - 1)
```

---

8. Writing the Recurrence Relation

The total time is:

```
Current work + recursive call time
```

Current work is:

```
1
```

Recursive call time is:

```
T(n - 1)
```

So:

```
T(n) = T(n - 1) + 1
```

This means:

> The time for input `n` is the time for input `n - 1`, plus one unit of work now.

---

9. Meaning of Each Part of the Recurrence

The recurrence is:

```
T(n) = T(n - 1) + 1
```

Part	Meaning
`T(n)`	Time taken for input size `n`
`T(n - 1)`	Time taken by the recursive call on the smaller input
`+1`	Constant work done in the current call

In simple words:

> The function does one small job now, then solves the same problem for `n - 1`.

---

10. What Is the Base Case?

The **base case** is the condition where recursion stops.

In the code:

```
if (n > 0)
```

The function continues only while `n > 0`.

When `n = 0`, the function stops.

So the base case is:

```
n = 0
```

In recurrence form, we write:

```
T(0) = 1
```

This means:

> When `n` reaches 0, the function stops in constant time.

11. Full Recurrence Relation

The recurrence has two parts.

Recursive case

This happens when `n > 0`:

```
T(n) = T(n - 1) + 1
```

Base case

This happens when `n = 0`:

```
T(0) = 1
```

So the full recurrence is:

```
T(n) = T(n - 1) + 1, for n > 0
T(0) = 1
```

---

12. Dry Run of `printNumber(5)`

Let us trace the function step by step.

Call	Condition	Work Done	Next Call
`printNumber(5)`	`5 > 0` is true	print `5`	`printNumber(4)`
`printNumber(4)`	`4 > 0` is true	print `4`	`printNumber(3)`
`printNumber(3)`	`3 > 0` is true	print `3`	`printNumber(2)`
`printNumber(2)`	`2 > 0` is true	print `2`	`printNumber(1)`
`printNumber(1)`	`1 > 0` is true	print `1`	`printNumber(0)`
`printNumber(0)`	`0 > 0` is false	stop	no call

Output:

```
5 4 3 2 1
```

---

13. Counting the Work

For `printNumber(5)`, the working calls are:

```
printNumber(5)
printNumber(4)
printNumber(3)
printNumber(2)
printNumber(1)
```

That is 5 working calls.

Each working call prints once.

So total work is about:

```
5
```

For general input `n`, the function does about `n` working calls.

So the time complexity is:

```
O(n)
```

---

14. Time Complexity

Time complexity tells us how the running time grows when the input gets bigger.

For this function:

```
void printNumber(int n) {
    if (n > 0) {
        cout << n << " ";
        printNumber(n - 1);
```

```
    }
}
```

The function reduces `n` by 1 each time:

```
n, n - 1, n - 2, n - 3, ..., 1, 0
```

So the number of calls grows linearly with `n`.

Therefore:

```
Time complexity = O(n)
```

This means:

- If `n` becomes bigger, the number of calls grows in a straight-line way.

---

15. Recursion Stack Space

Recursive functions use memory because each function call waits for the next function call to finish.

For `printNumber(5)`, the calls stack up like this:

```
printNumber(5)
  printNumber(4)
    printNumber(3)
      printNumber(2)
        printNumber(1)
          printNumber(0)
```

At one point, many calls are waiting.

For input `n`, about `n` calls can be waiting on the call stack.

So the recursion stack space is:

```
O(n)
```

---

16. Time Complexity vs Space Complexity

For this example:

```
Time complexity = O(n)
Space complexity = O(n)
```

Why time is `O(n)`

Because the function makes about `n` working calls.

Why space is `O(n)`

Because recursion creates about `n` stacked function calls before stopping.

---

17. Full Beginner-Friendly Program

```cpp
#include <iostream>
using namespace std;

// Prints numbers from n down to 1 using recursion
void printNumber(int n) {
    if (n > 0) {
        cout << n << " ";      // constant work: +1
        printNumber(n - 1);    // recursive call: T(n - 1)
    }
}

int main() {
    int n = 5;

    cout << "Output: ";
    printNumber(n);

    cout << endl;
    return 0;
}
```

Output:

```
Output: 5 4 3 2 1
```

Recurrence:

```
T(n) = T(n - 1) + 1
T(0) = 1
```

Time complexity:

```
O(n)
```

Space complexity:

```
O(n)
```

---

18. Important Terms in Phase 1

Recursion

A method where a function calls itself.

Example:

```
printNumber(n - 1);
```

Recursive Function

A function that contains a call to itself.

Example:

```
void printNumber(int n) {
    printNumber(n - 1);
}
```

Base Case

The condition where recursion stops.

In this example:

```
if (n > 0)
```

The function stops when:

```
n = 0
```

So:

```
T(0) = 1
```

Recursive Case

The part where the function calls itself again.

In this example:

```
printNumber(n - 1);
```

The recursive case gives:

```
T(n - 1)
```

`T(n)`

`T(n)` means the time taken for input size `n`.

It does not mean the printed output.

Extra Work

Extra work is the work done outside the recursive call.

In this example:

```
cout << n << " ";
```

This is constant work.

So we write:

```
+1
```

Recurrence Relation

A recurrence relation is the equation that describes the running time of recursive code.

For this example:

```
T(n) = T(n - 1) + 1
```

Time Complexity

Time complexity describes how the running time grows as `n` grows.

For this example:

```
O(n)
```

Recursion Stack Space

Recursive calls use stack memory.

For this example:

```
O(n)
```

---

19. Common Beginner Mistakes

Mistake 1: Forgetting the Base Case

A recursive function must have a stopping condition.

Bad idea:

```
void printNumber(int n) {
    cout << n << " ";
    printNumber(n - 1);
}
```

This function has no base case.

It may run forever or crash.

Correct version:

```
void printNumber(int n) {
    if (n > 0) {
        cout << n << " ";
        printNumber(n - 1);
    }
}
```

Mistake 2: Writing Only `T(n - 1)`

Some beginners write:

```
T(n) = T(n - 1)
```

This is incomplete.

Why?

Because the current call also does work:

```
cout << n << " ";
```

So the correct recurrence is:

```
T(n) = T(n - 1) + 1
```

Mistake 3: Confusing Output With Time Complexity

The output for `printNumber(5)` is:

```
5 4 3 2 1
```

But the time complexity is not the output.

The time complexity is based on how many operations or calls happen.

For `n = 5`, there are about 5 working calls.

For general `n`, there are about `n` working calls.

So:

```
O(n)
```

Mistake 4: Thinking `T(0)` Must Always Be 0

The base case can be written as:

```
T(0) = 1
```

This means the function stops in constant time.

In Big O notation, exact constants are not very important.

So whether the base case is written as `T(0) = 1` or another constant, it still represents constant time.

Mistake 5: Ignoring Stack Space

Some beginners only think about time.

But recursion also uses memory.

Because the function calls wait on the stack, this example uses:

```
O(n)
```

space.

---

20. Checklist for Writing a Recurrence Relation

When you see a recursive function, ask these questions.

Question 1: What is the base case?

For the example:

```
if (n > 0)
```

The function stops when:

```
n = 0
```

So:

```
T(0) = 1
```

Question 2: How many recursive calls are made?

The function has one recursive call:

```
printNumber(n - 1);
```

So there is one `T(...)` term.

Question 3: How does the input change?

The input changes from:

```
n
```

to:

```
n - 1
```

So the recursive part is:

```
T(n - 1)
```

Question 4: How much extra work happens outside recursion?

The function prints once:

```
cout << n << " ";
```

That is constant work.

So we write:

```
+1
```

Final Recurrence

Putting everything together:

```
T(n) = T(n - 1) + 1
T(0) = 1
```

---

21. Mini Practice Example

Look at this function:

```cpp
void countDown(int n) {
    if (n > 0) {
        cout << "Hello ";
        countDown(n - 1);
    }
}
```

Step 1: Find the base case

The function stops when:

```
n = 0
```

So:

```
T(0) = 1
```

Step 2: Count recursive calls

There is one recursive call:

```cpp
countDown(n - 1);
```

So the recursive part is:

```
T(n - 1)
```

Step 3: Find the extra work

This line prints once:

```cpp
cout << "Hello ";
```

So the extra work is:

```
+1
```

Final answer

```
T(n) = T(n - 1) + 1
T(0) = 1
```

Time complexity:

```
O(n)
```

Space complexity:

```
O(n)
```

---

22. Simple Way to Remember Phase 1

For this type of recursive function:

```
function(n) {
    do one small work;
    function(n - 1);
}
```

The recurrence is usually:

```
T(n) = T(n - 1) + 1
```

Because:

```
T(n - 1)
```

comes from the smaller recursive call, and:

```
+1
```

comes from the small work done now.

---

23. Phase 1 Final Summary

In Phase 1, we learned that recursive code can be written as a recurrence relation.

The main example was:

```
void printNumber(int n) {
    if (n > 0) {
        cout << n << " ";
        printNumber(n - 1);
    }
}
```

This function:

1. Prints the current value of `n`.
2. Calls itself with `n - 1`.
3. Stops when `n = 0`.

The recurrence relation is:

```
T(n) = T(n - 1) + 1
T(0) = 1
```

The time complexity is:

```
O(n)
```

The recursion stack space is:

```
O(n)
```

The most important lesson is:

> A recurrence relation describes how the running time of a recursive function depends on a smaller version of the same problem.

---

24. What You Should Be Able to Do After Phase 1

After this phase, you should be able to:

1. Identify a recursive function.
2. Find the base case.
3. Find the recursive call.
4. See how the input changes.
5. Count the extra work done outside recursion.
6. Write a simple recurrence relation.
7. Understand why `T(n) = T(n - 1) + 1` gives `O(n)` time.
8. Understand why recursion stack space is also `O(n)`.

---

25. Key Formula From Phase 1

```
T(n) = T(n - 1) + 1
T(0) = 1
```

Final result:

```
Time:  O(n)
Space: O(n)
```

This is the foundation for the next phase, where we solve this recurrence formally using successive substitution.